Modeling Memory and Register Banks

1 Introduction

Memory is commonly added to a design by instantiating a memory component from a design library. For simulation and for small storage structures, it is also useful to describe memory as an array of registers. The examples below introduce array declaration, initialization, RAM and ROM behavior, and register files. The code is presented as editable Verilog listings rather than screenshots.

2 Modeling Memories

2.1 Memory arrays and initialization

A memory array has an address dimension and a word-width dimension. In the example below, `mem` contains 1024 words, each 8 bits wide. The first address is zero and the last is 1023. Individual words can be accessed with array notation such as `mem[0]` and `mem[4]`.

```
module memory_model;
  reg [7:0] mem [0:1023];

  initial begin
    mem[0] = 8'b01001101;
    mem[4] = 8'b00000000;
  end
endmodule
```

An `initial` block is useful for simulation setup and for initializing small memories in supported synthesis flows. Larger data patterns are easier to maintain in a separate text file. Verilog provides `$readmemb` for binary data and `$readmemh` for hexadecimal data. The optional start and end addresses limit which portion of the array is loaded; when omitted, the available memory range is read.

```
module memory_model;
  reg [7:0] mem [0:1023];

  initial begin
    $readmemh("mem.dat", mem);
  end
endmodule
```

For a binary file and a bounded address range, use this call instead:

```
$readmemb("mem.dat", mem, 50, 200);
```

The file name is interpreted relative to the simulator's working directory. Each file should contain values in the format expected by the selected function; the data width should also match the declared word width.

2.2 Single-port RAM

A single-port RAM uses one address and one data path for both reading and writing. The `cs` signal selects the memory, while `rd` and `wr` request a read or a write. The following model uses a shared, tri-state data bus. A synchronous read updates `d_out` on a rising clock edge, and a write stores the bus value on a rising edge. Read and write controls should be mutually exclusive.

```
module ram_1 (addr, data, clk, rd, wr, cs);
  input [9:0] addr;
  input clk, rd, wr, cs;
  inout [7:0] data;

  reg [7:0] mem [0:1023];
  reg [7:0] d_out;

  assign data = (cs && rd && !wr) ? d_out : 8'bz;

  always @(posedge clk)
    if (cs && wr && !rd)
      mem[addr] <= data;

  always @(posedge clk)
    if (cs && rd && !wr)
      d_out <= mem[addr];
endmodule
```

The next version illustrates an asynchronous read and write model. Its behavior can be useful in a simulation, but asynchronous writes are not a portable way to infer hardware RAM. For synthesis, use a clocked write process in the form required by the target device and tool. Many tools also handle `inout` ports inconsistently, so separate input and output data signals are often preferable.

```
module ram_2 (addr, data, rd, wr, cs);
  input [9:0] addr;
  input rd, wr, cs;
  inout [7:0] data;

  reg [7:0] mem [0:1023];
  reg [7:0] d_out;

  assign data = (cs && rd && !wr) ? d_out : 8'bz;

  always @(addr or data or rd or wr or cs)
    if (cs && wr && !rd)
```

```
      mem[addr] = data;

  always @(addr or rd or wr or cs)
    if (cs && rd && !wr)
      d_out = mem[addr];
endmodule
```

2.3 ROM and explicit bus buffering

A ROM can be described with a case statement that maps each address to a constant word. This example defines three entries and returns zero for the remaining addresses. The default assignment ensures that the combinational process assigns the output for every input combination.

```
module rom (addr, data, rd_en, cs);
  input [2:0] addr;
  input rd_en, cs;
  output reg [7:0] data;

  always @* begin
    data = 8'd0;
    if (rd_en && cs) begin
      case (addr)
        3'd0: data = 8'd22;
        3'd1: data = 8'd45;
        3'd7: data = 8'd12;
        default: data = 8'd0;
      endcase
    end
  end
endmodule
```

If a bidirectional external bus is required, separate tri-state buffers can control the directions of travel. In practice, a top-level interface that uses separate input and output signals is generally easier to simulate and synthesize.

The tri-state assignments below illustrate the direction control. Keep the read and write enables mutually exclusive so that two sources do not drive the shared bus at the same time.

```
tri [7:0] Bus;
wire [7:0] data_out, data_in;
assign Bus = read ? data_out : 8'hzz;
assign data_in = write ? Bus : 8'hzz;
```

2.4 Parameterized memory model and test bench

Parameters make the word width, address width, and capacity easier to adjust. This example retains the separate input and output buses used by the original note. It is an asynchronous, simulation-oriented model; use a clocked write process when modeling a hardware RAM intended for synthesis.

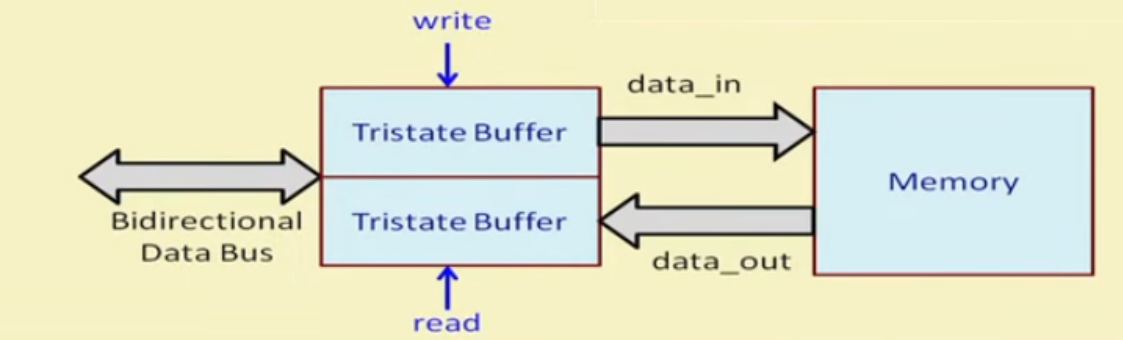


Figure 1: Separate tri-state paths for writes to and reads from memory.

```
module ram_3 (data_out, data_in, addr, wr, cs);
  parameter addr_size = 10;
  parameter word_size = 8;
  parameter memory_size = 1024;

  input [addr_size-1:0] addr;
  input [word_size-1:0] data_in;
  input wr, cs;
  output [word_size-1:0] data_out;

  reg [word_size-1:0] mem [0:memory_size-1];

  assign data_out = mem[addr];

  always @*
    if (cs && wr)
      mem[addr] = data_in;
endmodule
```

The test bench below writes a simple pattern to all 1024 addresses and then prints values from 20 pseudo-random addresses. It initializes the signals explicitly and samples the asynchronous output after a short delay.

```
module RAM_test;
  reg [9:0] address;
  reg [7:0] data_in;
  reg write, select;
  wire [7:0] data_out;
  integer k, myseed;

  ram_3 RAM (data_out, data_in, address, write, select);

  initial begin
    address = 0;
    data_in = 0;
    write = 0;
    select = 0;
    myseed = 35;
```

```
16
17      for (k = 0; k < 1024; k = k + 1) begin
18        address = k;
19        data_in = (k + k) % 256;
20        write = 1;
21        select = 1;
22        #2;
23        write = 0;
24        select = 0;
25        #2;
26      end
27
28      repeat (20) begin
29        address = ($random(myseed) & 32'h7fffffff) % 1024;
30        write = 0;
31        select = 1;
32        #1;
33        $display("Address: %5d, Data: %4d", address, data_out);
34        #1;
35        select = 0;
36      end
37    end
38  endmodule
```

A sample of the simulation output shown in the source note is reproduced below. The exact pseudo-random sequence can vary between simulators.

```
Address:  960, Data:  128
Address:  482, Data:  196
Address:  693, Data:  106
Address:  246, Data:  236
Address:  228, Data:  200
Address:  148, Data:   40
Address:  767, Data:  254
Address:  355, Data:  198
Address:  259, Data:    6
Address:   21, Data:   42
Address:  872, Data:  208
Address:  758, Data:  236
Address:  193, Data:  130
Address:  909, Data:   26
Address:  632, Data:  240
Address:  719, Data:  158
Address:  214, Data:  172
Address:   67, Data:  134
Address:  908, Data:   24
```

3 Modeling Register Banks

A register bank (or register file) is a collection of addressable registers. Many designs use two read ports and one write port so that two values can be read while another register is selected for

writing. The diagram shows a 32-word bank with 32-bit data words and 5-bit register indexes.

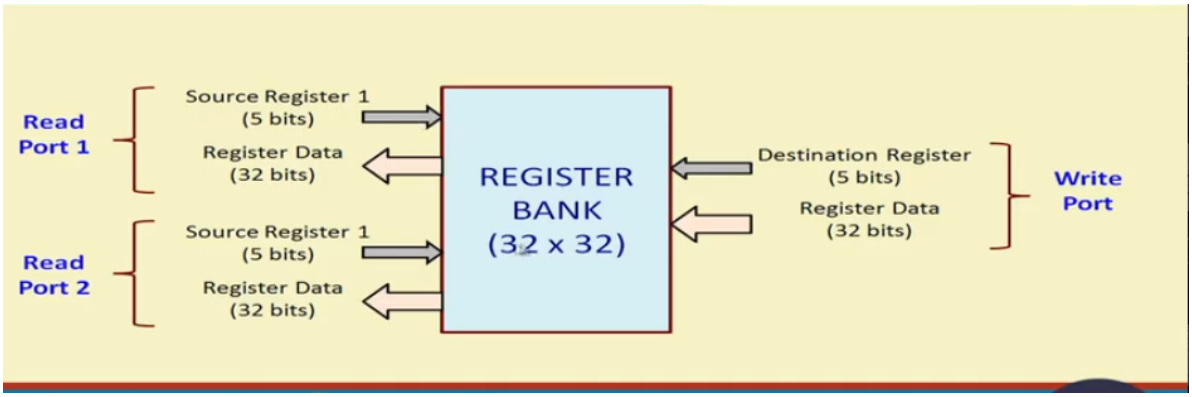


Figure 2: Two independent read ports and one write port on a register bank.

3.1 Explicit registers

For a very small bank, each register can be declared separately. The two combinational `case` statements select the read data, while the clocked process updates the selected destination register. This approach is straightforward but becomes cumbersome as the bank grows.

```
module regbank_v1 (rdData1, rdData2, wrData, sr1, sr2, dr, write, clk);
  input clk, write;
  input [1:0] sr1, sr2, dr;
  input [31:0] wrData;
  output reg [31:0] rdData1, rdData2;
  reg [31:0] R0, R1, R2, R3;

  always @* begin
    case (sr1)
      0: rdData1 = R0;
      1: rdData1 = R1;
      2: rdData1 = R2;
      3: rdData1 = R3;
      default: rdData1 = 32'hxxxxxxxx;
    endcase
  end

  always @* begin
    case (sr2)
      0: rdData2 = R0;
      1: rdData2 = R1;
      2: rdData2 = R2;
      3: rdData2 = R3;
      default: rdData2 = 32'hxxxxxxxx;
    endcase
  end

  always @(posedge clk) begin
    if (write) begin
```

```
      case (dr)
        0: R0 <= wrData;
        1: R1 <= wrData;
        2: R2 <= wrData;
        3: R3 <= wrData;
      endcase
    end
  end
endmodule
```

The same small bank can express the read multiplexers with conditional operators. This changes the syntax, not the number of registers or the read/write behavior.

```
module regbank_v2 (rdData1, rdData2, wrData, sr1, sr2, dr, write, clk);
  input clk, write;
  input [1:0] sr1, sr2, dr;
  input [31:0] wrData;
  output [31:0] rdData1, rdData2;
  reg [31:0] R0, R1, R2, R3;

  assign rdData1 = (sr1 == 0) ? R0 :
                   (sr1 == 1) ? R1 :
                   (sr1 == 2) ? R2 : R3;
  assign rdData2 = (sr2 == 0) ? R0 :
                   (sr2 == 1) ? R1 :
                   (sr2 == 2) ? R2 : R3;

  always @(posedge clk) begin
    if (write) begin
      case (dr)
        0: R0 <= wrData;
        1: R1 <= wrData;
        2: R2 <= wrData;
        3: R3 <= wrData;
      endcase
    end
  end
endmodule
```

3.2 Array-based register files

Declaring the bank as an array scales more naturally. A 5-bit index selects one of 32 words. The two continuous assignments provide asynchronous reads; the clocked process performs a synchronous write.

```
module regbank_v3 (rdData1, rdData2, wrData, sr1, sr2, dr, write, clk);
  input clk, write;
  input [4:0] sr1, sr2, dr;
  input [31:0] wrData;
  output [31:0] rdData1, rdData2;

  reg [31:0] regfile [0:31];
```

```
8
9    assign rdData1 = regfile[sr1];
10   assign rdData2 = regfile[sr2];
11
12   always @(posedge clk)
13     if (write)
14       regfile[dr] <= wrData;
15 endmodule
```

The next version adds a synchronous, active-high reset that clears all 32 entries. Resetting every word is convenient in a model, but large resettable arrays may not map to dedicated memory resources in every target technology.

```
1  module regbank_v4 (rdData1, rdData2, wrData, sr1, sr2, dr,
2                     write, reset, clk);
3    input clk, write, reset;
4    input [4:0] sr1, sr2, dr;
5    input [31:0] wrData;
6    output [31:0] rdData1, rdData2;
7    integer k;
8
9    reg [31:0] regfile [0:31];
10
11   assign rdData1 = regfile[sr1];
12   assign rdData2 = regfile[sr2];
13
14   always @(posedge clk) begin
15     if (reset) begin
16       for (k = 0; k < 32; k = k + 1)
17         regfile[k] <= 32'b0;
18     end else if (write) begin
19       regfile[dr] <= wrData;
20     end
21   end
22 endmodule
```

These examples model a generic register file. If a design requires one architecturally fixed register (for example, a register that must always read as zero), implement that behavior explicitly rather than allowing writes to change the corresponding array entry.

3.3 Register-file test bench

The test bench below generates a clock, asserts reset, writes values to all registers, and then reads pairs of registers. Driving inputs on the falling edge avoids changing them at the same time as the design's rising-edge update.

```
1  module regfile_test;
2    reg [4:0] sr1, sr2, dr;
3    reg [31:0] wrData;
4    reg write, reset, clk;
5    wire [31:0] rdData1, rdData2;
```

```
  integer k;

  regbank_v4 REG
    (rdData1, rdData2, wrData, sr1, sr2, dr, write, reset, clk);

  initial clk = 0;
  always #5 clk = ~clk;

  initial begin
    $dumpfile("regfile.vcd");
    $dumpvars(0, regfile_test);
    sr1 = 0;
    sr2 = 0;
    dr = 0;
    wrData = 0;
    write = 0;
    reset = 0;

    #1 reset = 1;
    #10 reset = 0;

    for (k = 0; k < 32; k = k + 1) begin
      @(negedge clk);
      dr = k;
      wrData = 10 * k;
      write = 1;
      @(negedge clk);
      write = 0;
    end

    for (k = 0; k < 32; k = k + 2) begin
      @(negedge clk);
      sr1 = k;
      sr2 = k + 1;
      #1;
      $display("reg[%0d] = %0d, reg[%0d] = %0d",
               sr1, rdData1, sr2, rdData2);
    end

    #10 $finish;
  end
endmodule
```